Processingをはじめよう

第5章

反応

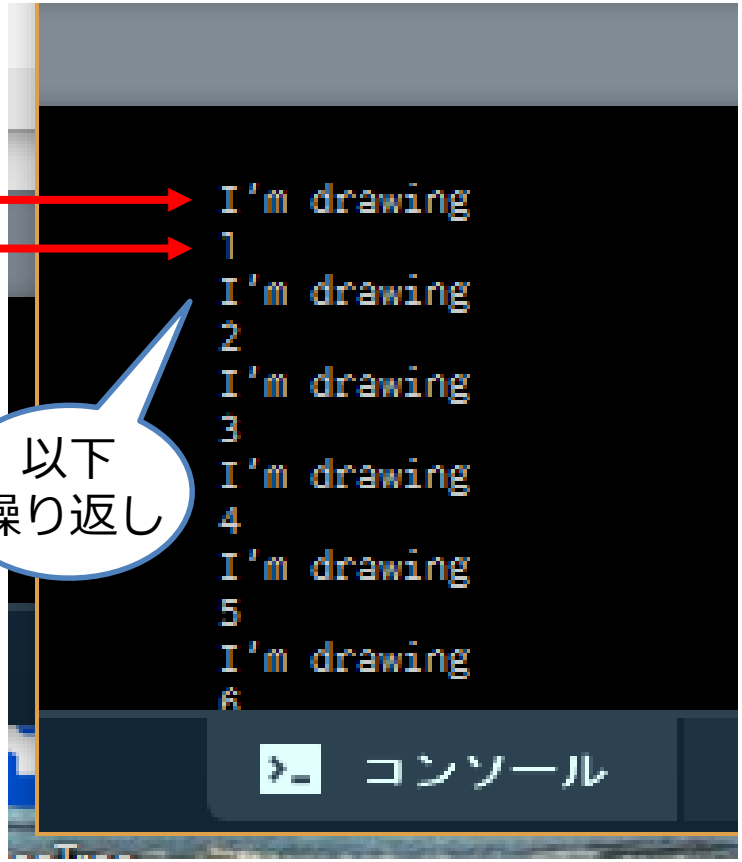
目次

- 繰り返されるdrawと一度だけのsetup
- 追いかける
- クリック
- カーソルの位置
- キーボードからの入力
- マッピング
- Robot 3: Response

繰り返されるdrawと一度だけのsetup

- Example 5-1 draw()関数

```
void draw() {
  println("I'm drawing");
  println(frameCount);
}
```

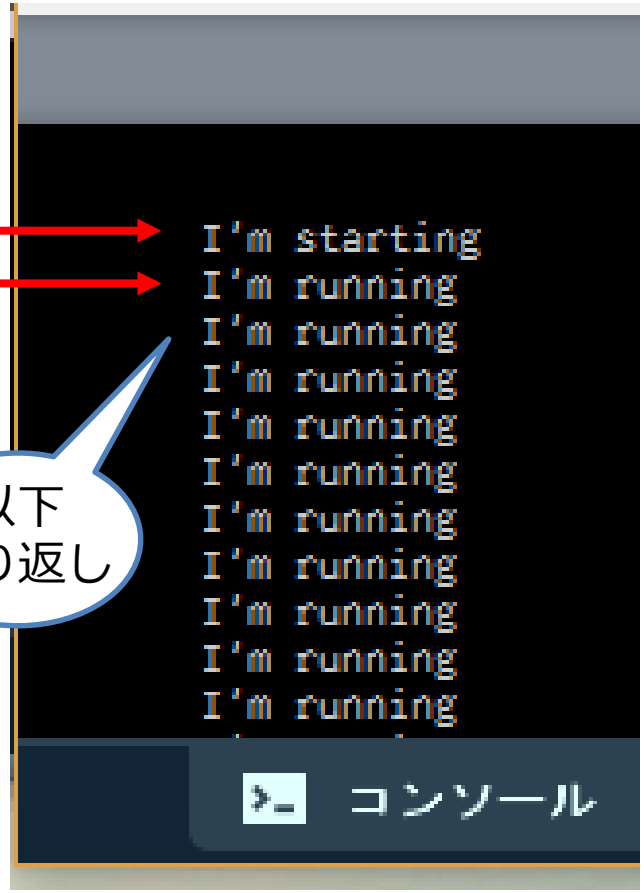


- draw()関数
  - ブロック内のコードを繰り返し実行
- 変数frameCount
  - 教科書 P225
  - プログラム開始から何フレーム表示したかを保持する変数（何回繰り返したか）

繰り返されるdrawと一度だけのsetup

- Example 5-2 setup()関数

```
void setup() {
  println("I'm starting");
}

void draw() {
  println("I'm running");
}
```

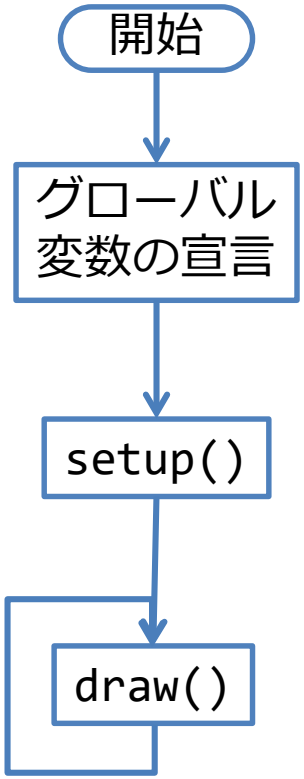


- setup()関数
  - ブロック内のコードを開始時1回だけ実行

setup()とdraw()の使い方

- Processingがコードを実行する順番

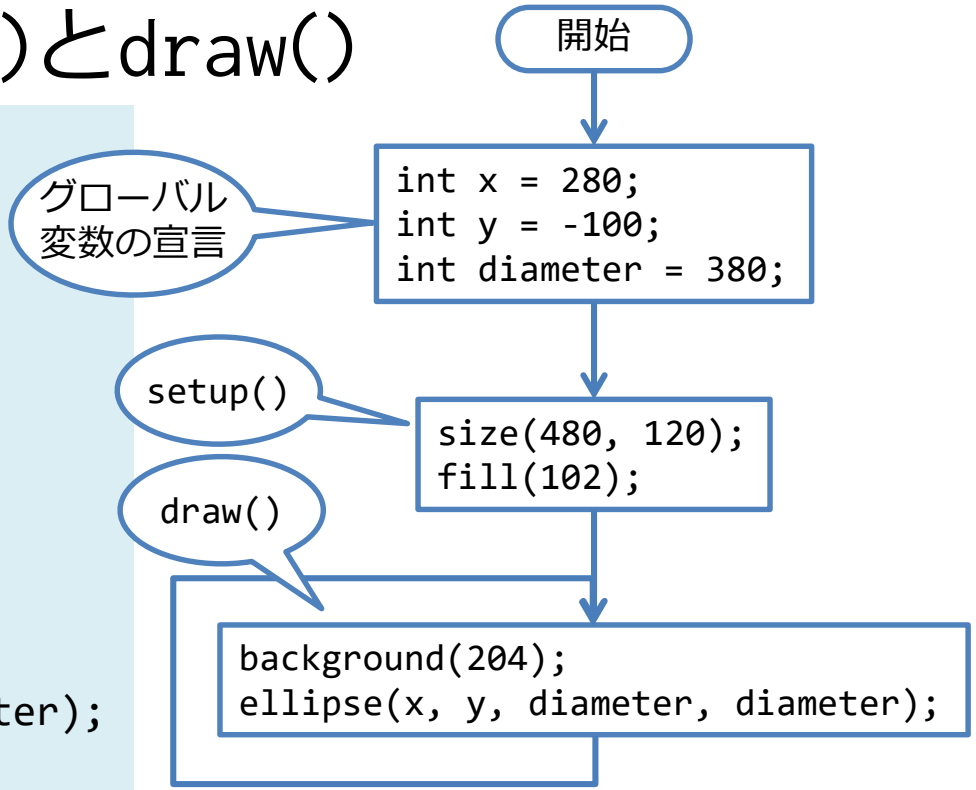
1. setup()とdraw()の外側で宣言された変数（グローバル変数）を作成
2. setup()内のコードを一度だけ実行
3. draw()内のコードを繰り返し実行

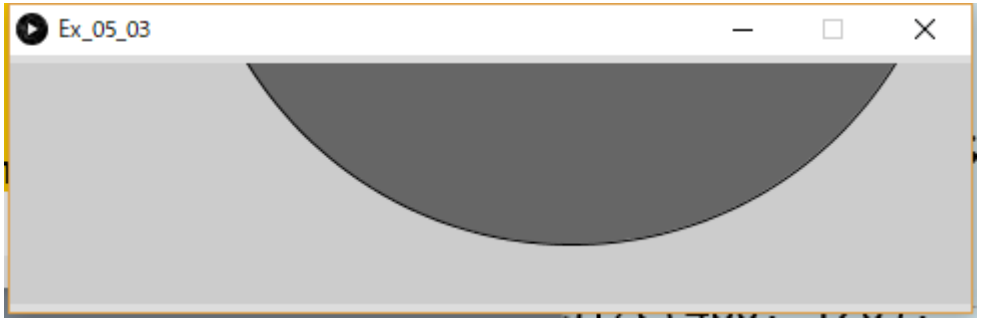


繰り返されるdrawと一度だけのsetup

- Example 5-3 setup()とdraw()

```
int x = 280;
int y = -100;
int diameter = 380;

void setup() {
  size(480, 120);
  fill(102);
}

void draw() {
  background(204);
  ellipse(x, y, diameter, diameter);
}
```

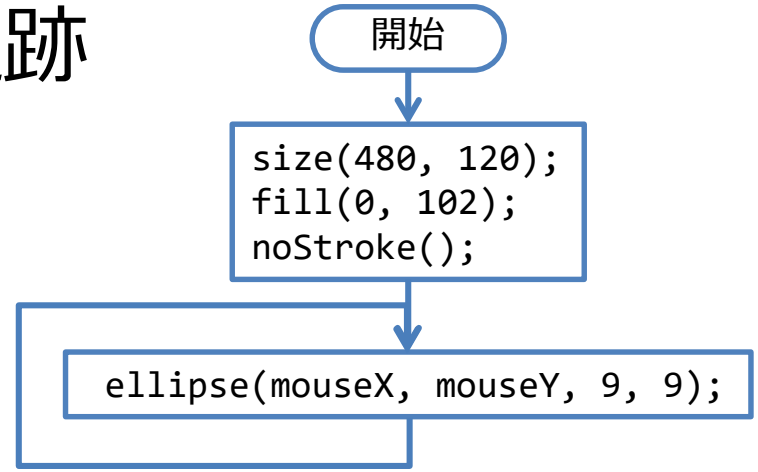




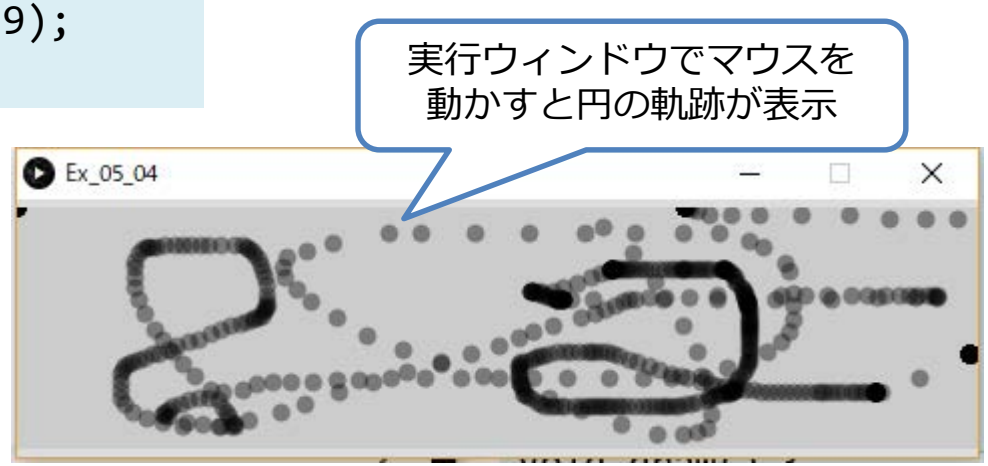
追いかける

- Example 5-4 マウスを追跡

```
void setup() {
  size(480, 120);
  fill(0, 102);    //塗り色、透明度指定
  noStroke();      //輪郭線を消す
}

void draw() {
  ellipse(mouseX, mouseY, 9, 9);
}
```



- mouseX , mouseY
  - 教科書P256
  - マウスのカーソル位置を表す変数

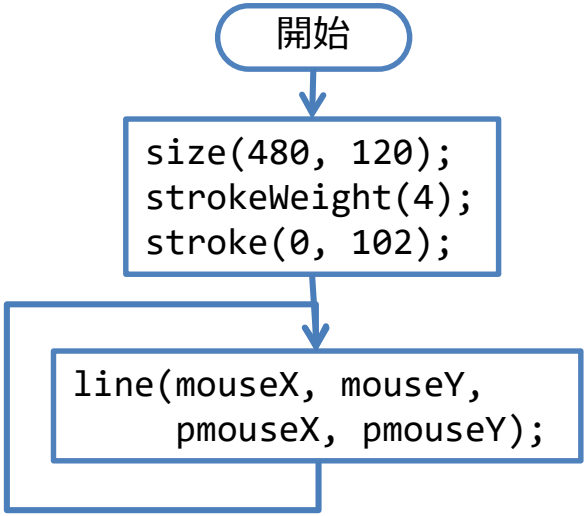


- Example 5-5 背景を塗ると・・・

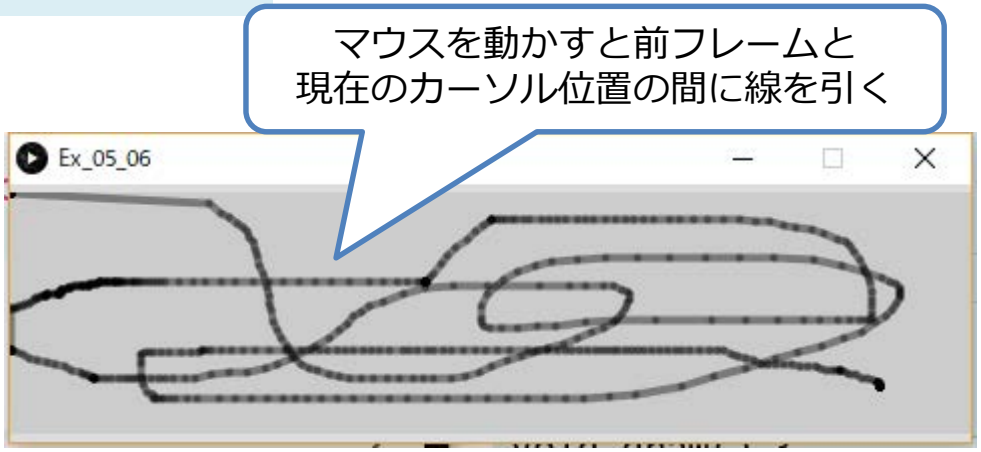
追いかける

- Example 5-6 連続的に描く

```
void setup() {
  size(480, 120);
  strokeWeight(4); //輪郭線の太さ4
  stroke(0, 102);  //輪郭線の色、透明度設定
}

void draw() {
  line(mouseX, mouseY, pmouseX, pmouseY);
}
```



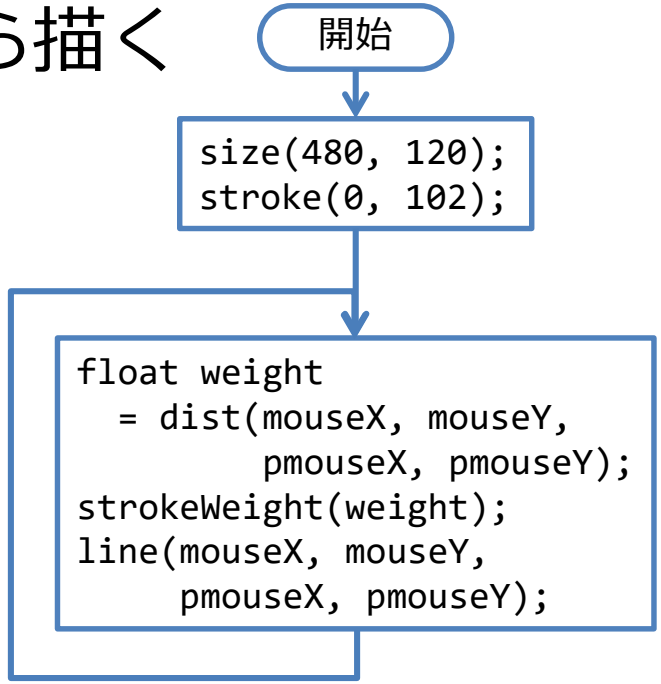
- pmouseX , pmouseY
  - 教科書P256
  - 前フレームの
    マウスのカーソル
    位置を表す変数



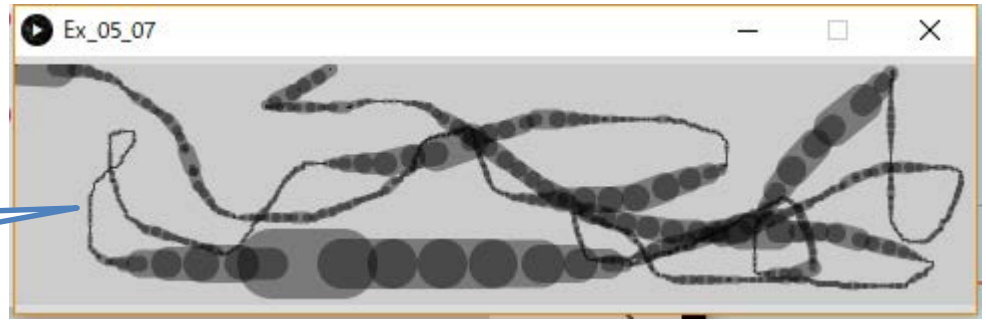
追いかける

- Example 5-7 太さを変えながら描く

```
void setup() {
  size(480, 120);
  stroke(0, 102);
}

void draw() {
  float weight = dist(mouseX, mouseY,
                      pmouseX, pmouseY);
  strokeWeight(weight);
  line(mouseX, mouseY, pmouseX, pmouseY);
}
```



- dist(x1, y1, x2, y2)
  - 教科書P230
  - (x1, y1)と(x2, y2)の間の距離を求める

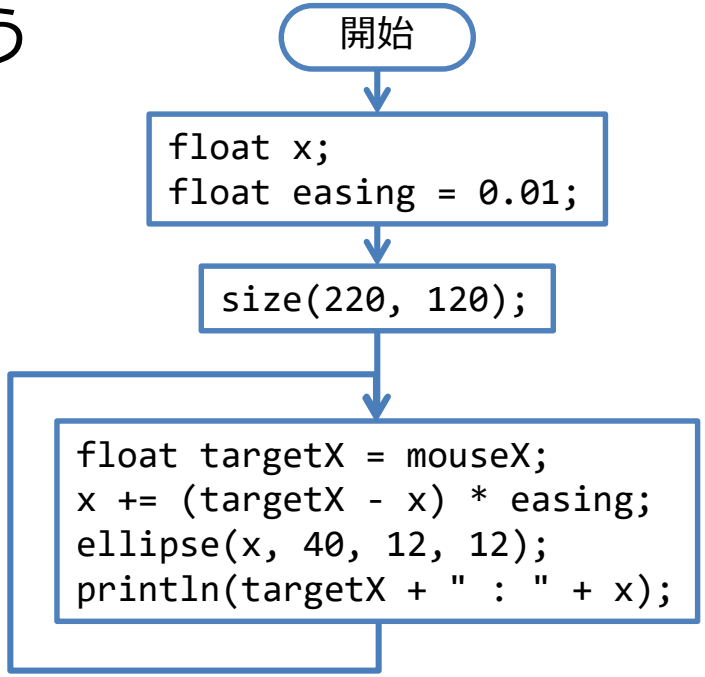


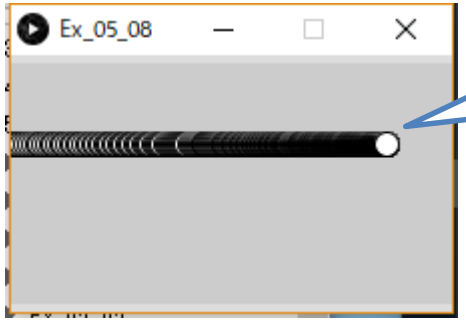
2点が遠い（マウスを早く移動する）と太い線

追いかける

- Example 5-8 ゆっくり行こう

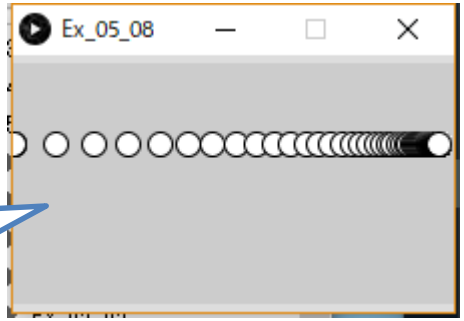
```
float x;
float easing = 0.01;

void setup() {
  size(220, 120);
}

void draw() {
  float targetX = mouseX;
  x += (targetX - x) * easing;  //イージング
  ellipse(x, 40, 12, 12);
  println(targetX + " : " + x);
}
```





easing = 0.01
マウスにゆっくり追いつく

easing = 0.1
マウスにすぐ追いつく



イージング

- イージング(Easing)
  マウスの急激な動きを緩和する
  - 変数　`float easing = 0.01;`
  - 計算部分

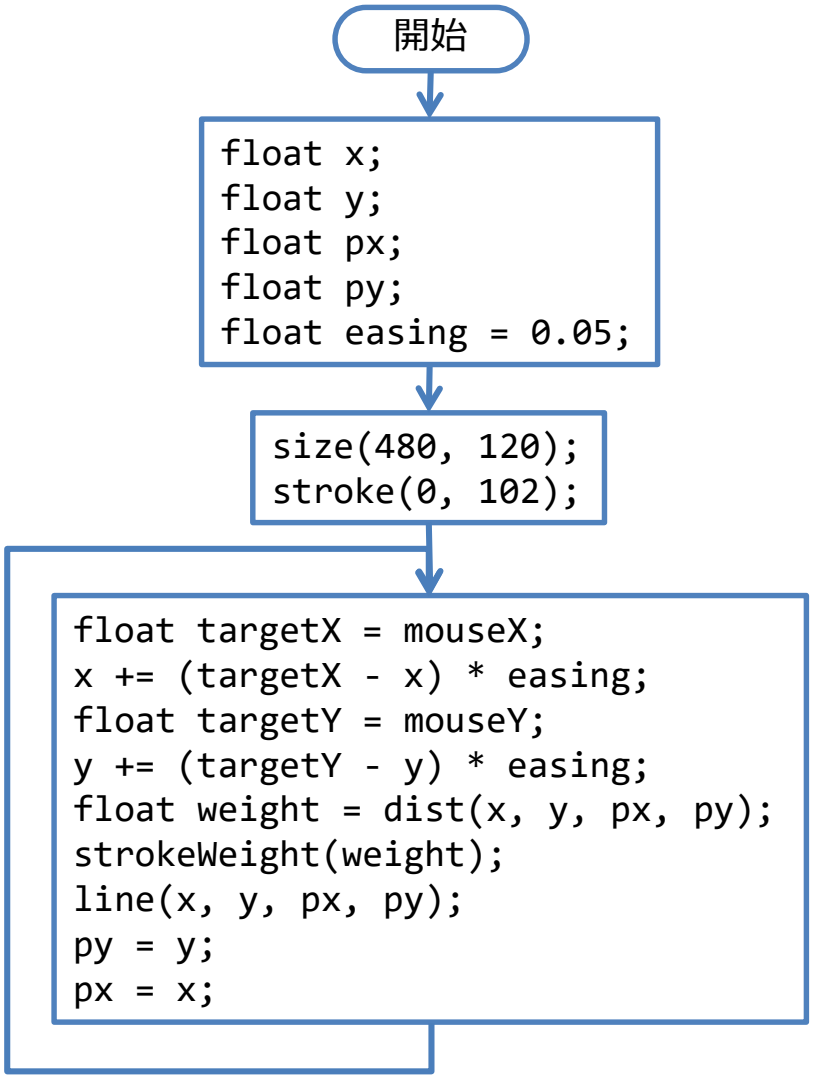
    ```
    x += (targetX - x) * easing;
    ```

x	現在値（円を表示する位置）
targetX	目標値（マウスの位置）
easing	緩和するための係数　0～1の間
  - 係数を変えて、動きがどう変化するか確認

追いかける

- Example 5-9 イージングで線を滑らかに

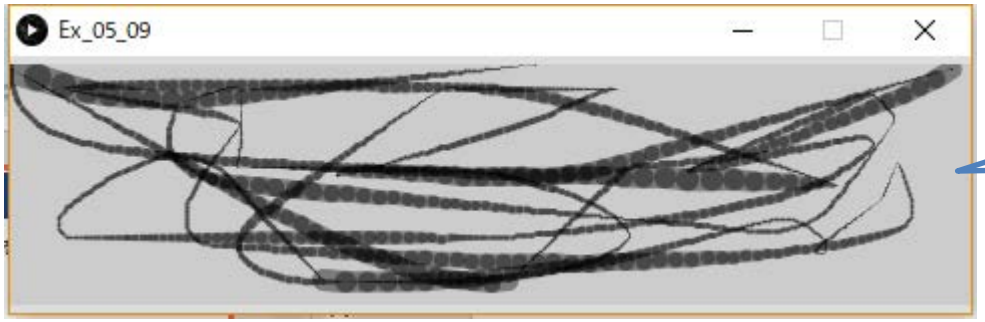
```
float x;    //x座標
float y;    //y座標
float px;   //前フレームのx座標
float py;   //前フレームのy座標
float easing = 0.05;

void setup() {
  size(480, 120);
  stroke(0, 102);
}

void draw() {
  float targetX = mouseX;
  x += (targetX - x) * easing;
  float targetY = mouseY;
  y += (targetY - y) * easing;
  float weight = dist(x, y, px, py);
  strokeWeight(weight);
  line(x, y, px, py);
  py = y;   //y座標を前フレームのy座標に代入
  px = x;   //x座標を前フレームのx座標に代入
}
```



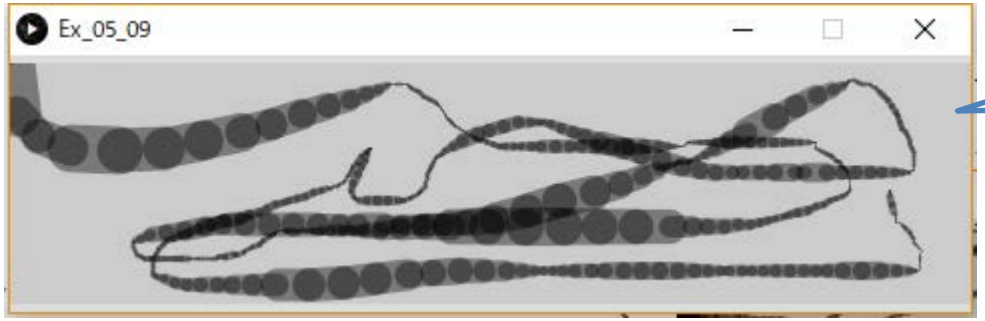
追いかける

- Example 5-7 と 5-8 を合わせた処理
  - x座標のイージング、y座標のイージング
  - (x, y)と(px, py)の距離計算、線の太さ設定
  - (x, y)と(px, py)の2点に線を引く
  - (px, py)に(x, y)を代入

次回に
「1フレーム前の」
x,y座標として使うため



easing = 0.05



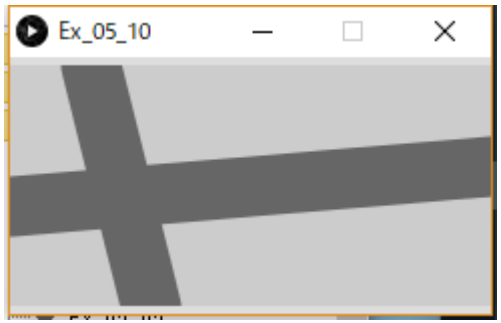
easing = 0.5

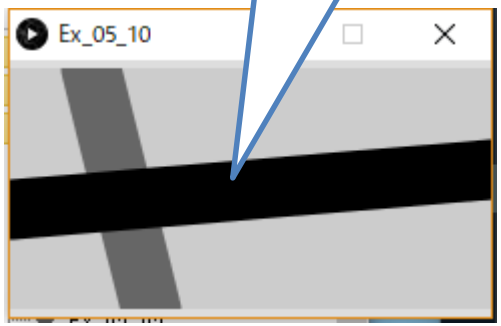
クリック

- Example 5-10 マウスをクリック

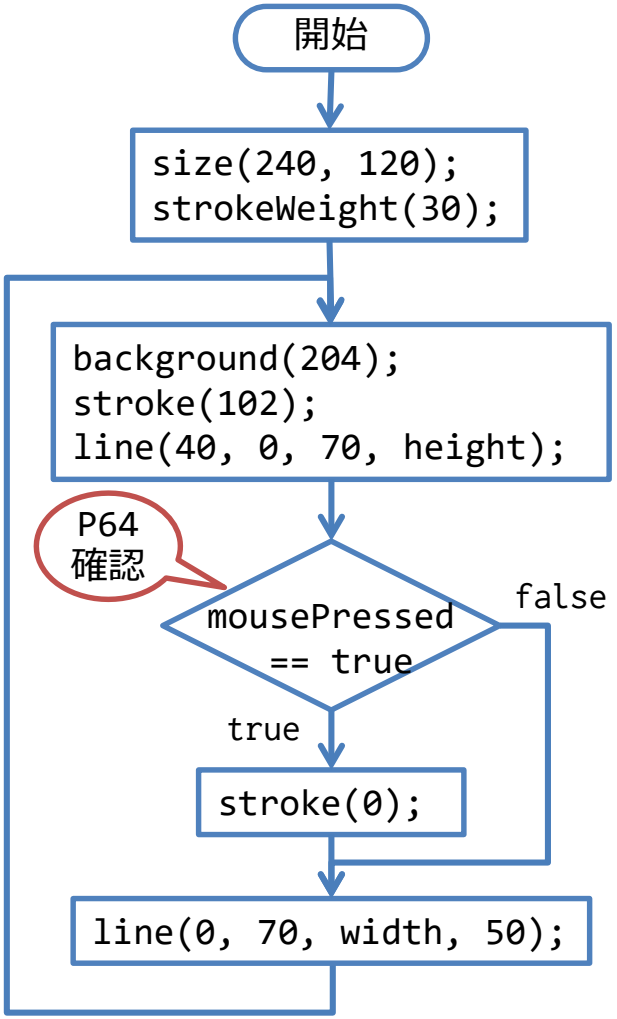
```
void setup() {
  size(240, 120);
  strokeWeight(30);
}

void draw() {
  background(204);
  stroke(102);
  line(40, 0, 70, height);
  if (mousePressed == true) {
    stroke(0);
  }
  line(0, 70, width, 50);
}
```

- mousePressed変数
  - 教科書 P256
  - クリックするとtrue しないとfalse





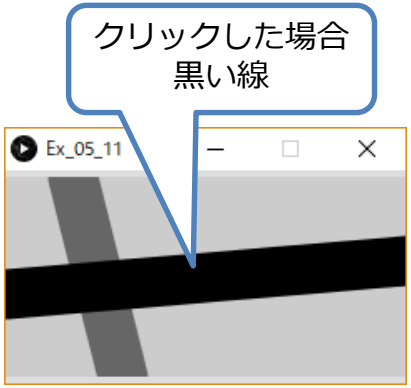
クリックした場合
黒い線

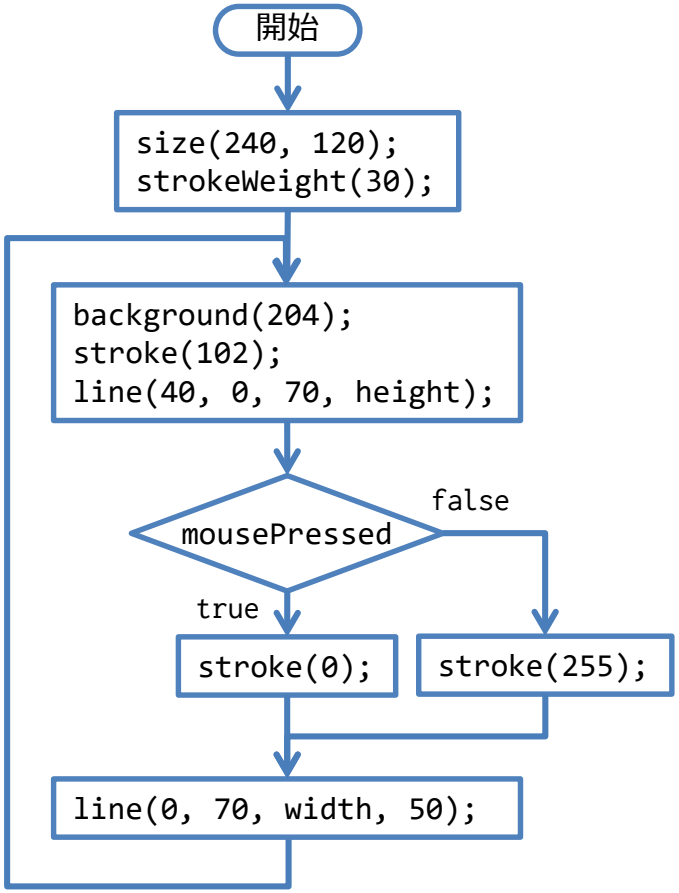


クリック

- Example 5-11 クリックされていないことを検出する

```
void setup() {
  size(240, 120);
  strokeWeight(30);
}

void draw() {
  background(204);
  stroke(102);
  line(40, 0, 70, height);
  if (mousePressed) {
    stroke(0);
  } else {
    stroke(255);
  }
  line(0, 70, width, 50);
}
```





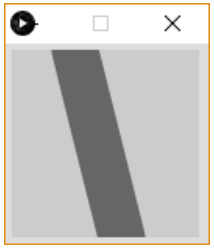
クリック

- Example 5-12 複数のマウスボタン

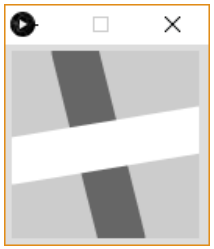
```
void setup() {
  size(120, 120);
  strokeWeight(30);
}

void draw() {
  background(204);
  stroke(102);
  line(40, 0, 70, height);
  if (mousePressed) {
    if (mouseButton == LEFT) {
      stroke(255);
    } else {
      stroke(0);
    }
    line(0, 70, width, 50);
  }
}
```

mouseButton 変数 P256
値：LEFT, RIGHT, CENTER

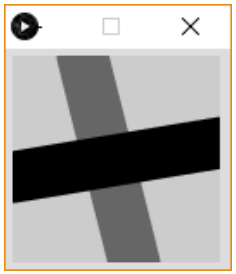
クリックなし

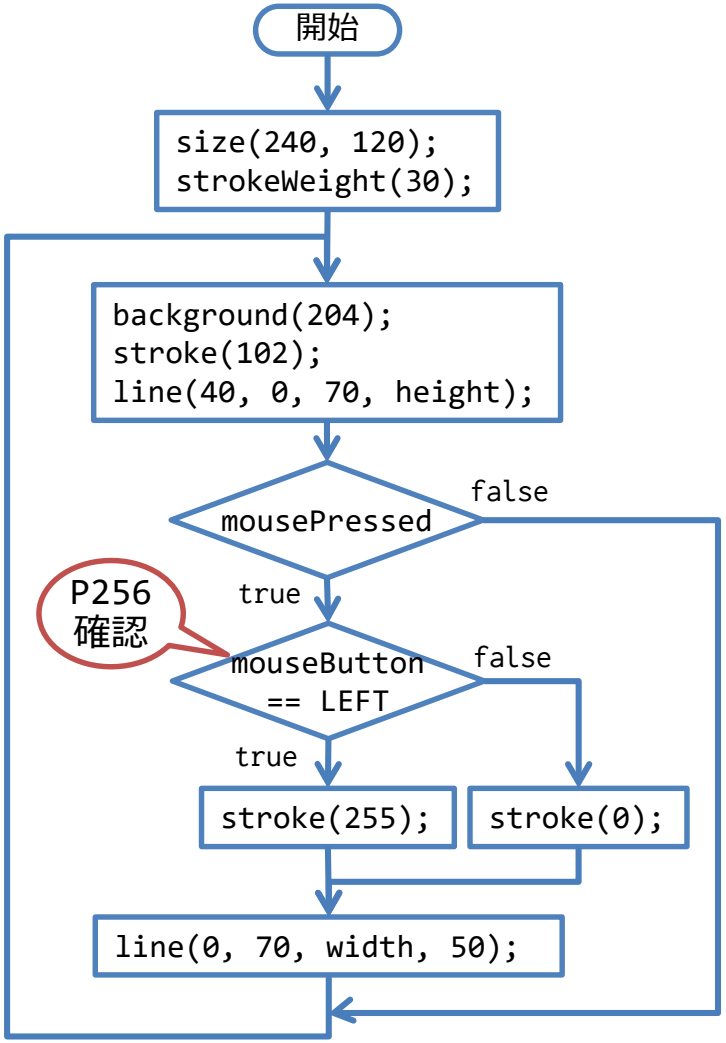


左クリック



左以外をクリック



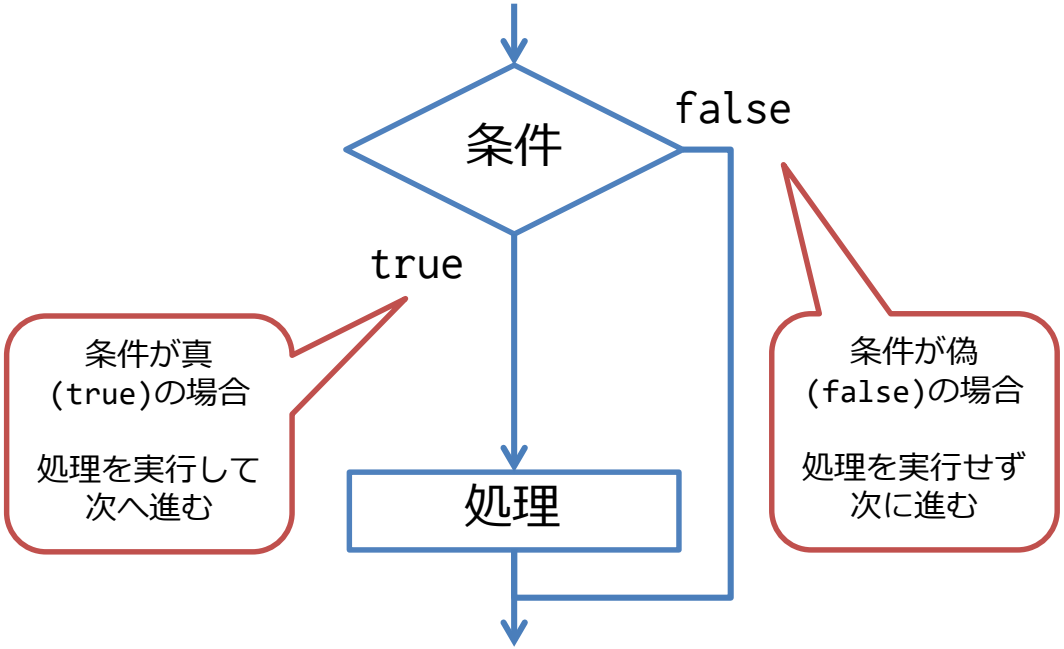


条件とフローチャート ①

- if文

```
if (条件) {
  処理
}
```

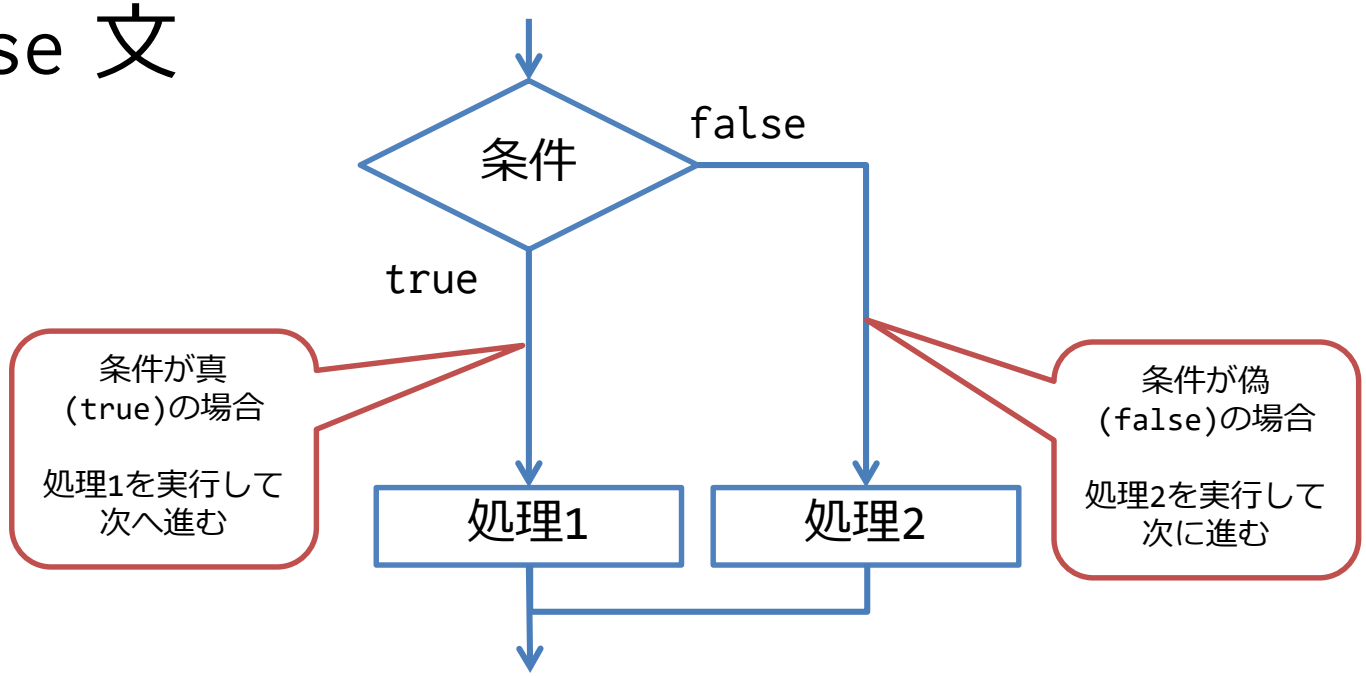
  - 条件 P46～47 参照
    - 2つの値を比べる式
  - 処理 実行する命令
    - 逐次
    - 条件分岐
    - 繰り返し
    - 関数の呼び出し
    - 複数行でも可



条件とフローチャート ②

- if ～ else 文

```
if (条件) {
  処理1
} else {
  処理2
}
```

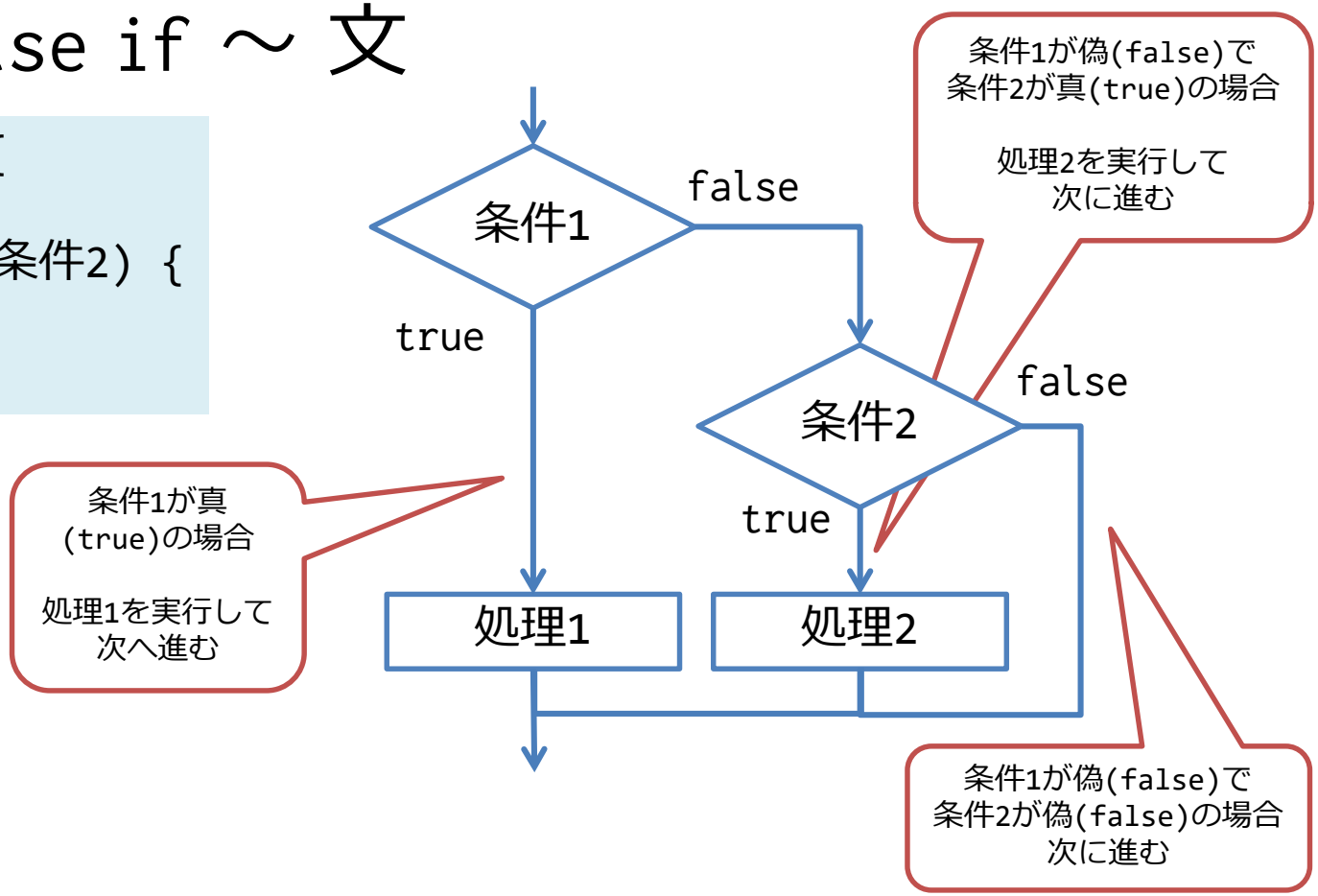


– 条件が真と偽の場合で異なるの処理を実行

条件とフローチャート ③

- if ～ else if ～ 文

```
if (条件1) {
  処理1
} else if (条件2) {
  処理2
}
```

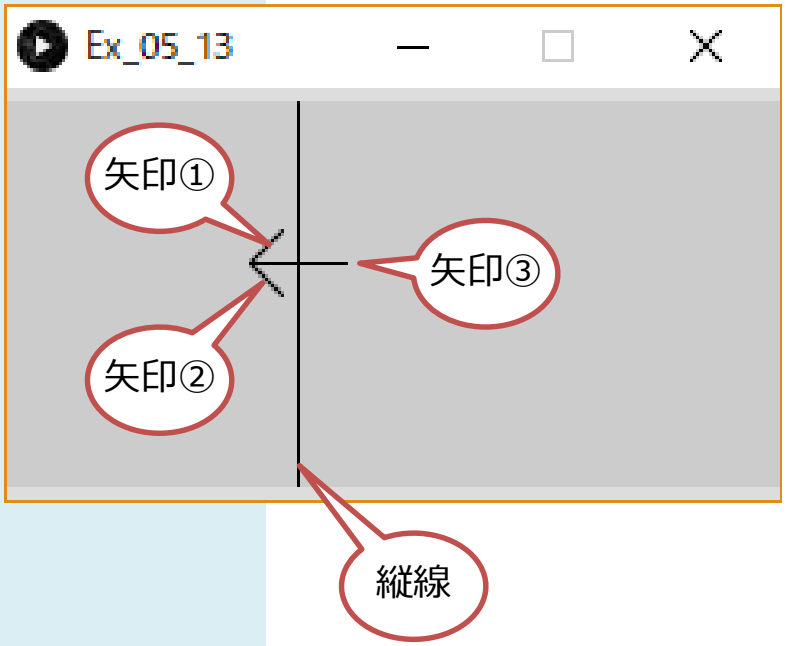


– ②の処理2に①のフローチャートが入った形

カーソル

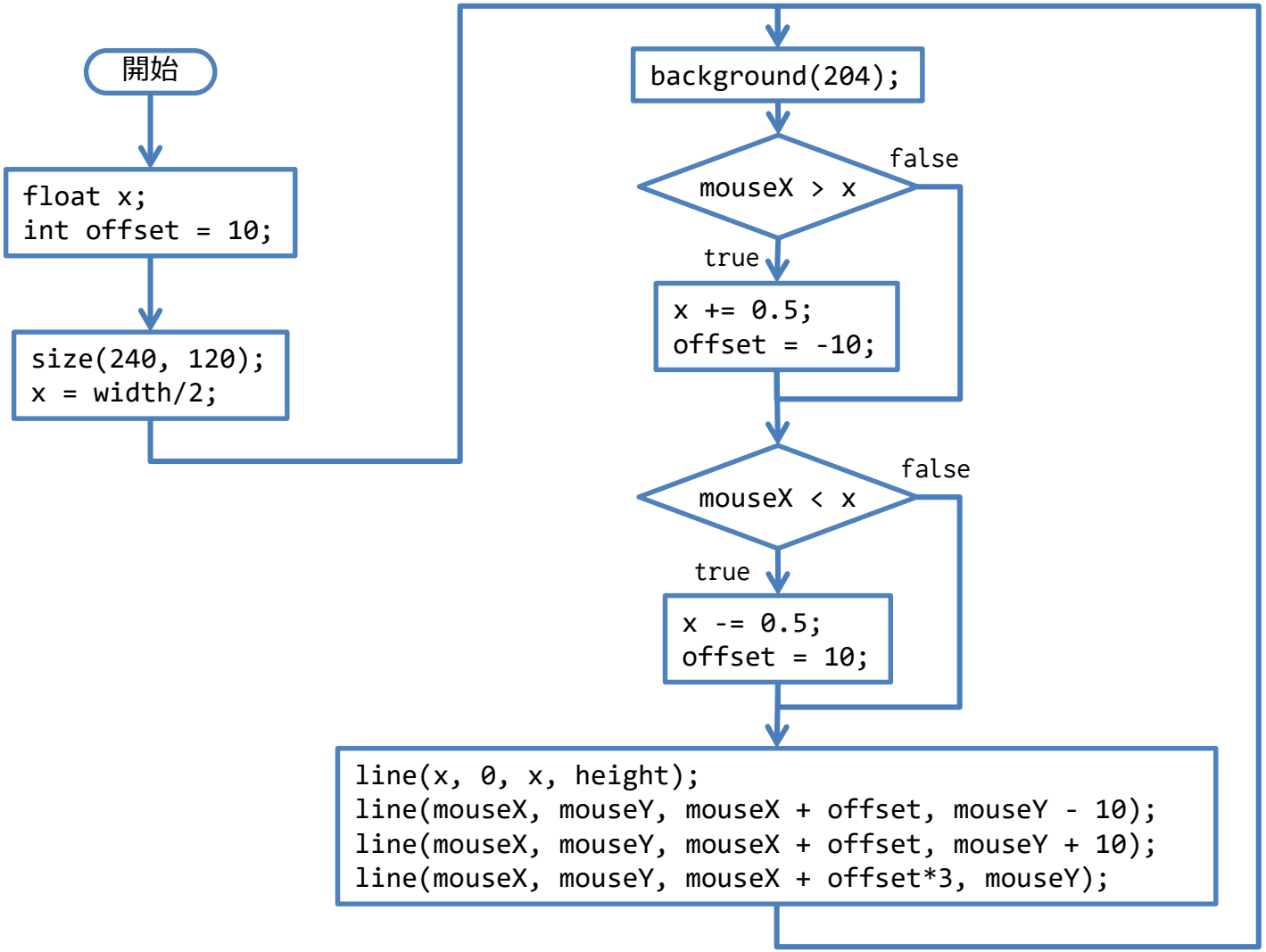
- Example 5-13 カーソルを探せ

```
float x;           // 縦線のx座標
int offset = 10;   // 矢印の向き・大きさを表す変数

void setup() {
  size(240, 120);
  x = width/2;     // 縦線の初期位置は中央
}

void draw() {
  background(204);
  if (mouseX > x) { // マウスが縦線より右にある場合
    x += 0.5;       // 縦線を右に0.5移動
    offset = -10;   // 矢印を右向き
  }
  if (mouseX < x) { // マウスが縦線より左にある場合
    x -= 0.5;       // 縦線を左に0.5移動
    offset = 10;    // 矢印を左向き
  }
  line(x, 0, x, height);  //縦線
  line(mouseX, mouseY, mouseX + offset, mouseY - 10);  //矢印①
  line(mouseX, mouseY, mouseX + offset, mouseY + 10);  //矢印②
  line(mouseX, mouseY, mouseX + offset*3, mouseY);     //矢印③
}
```



カーソル

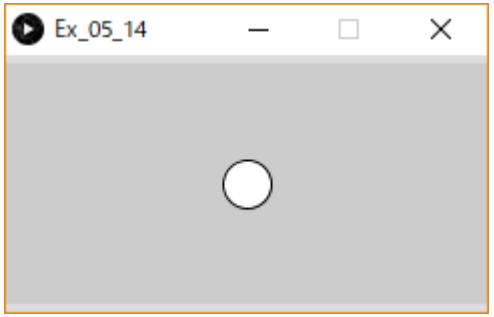
- Example 5-13 カーソルを探せ

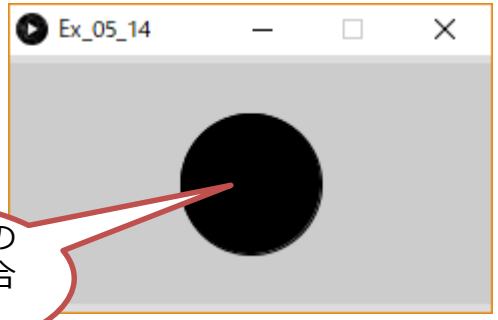


カーソル

- Example 5-14 円の境界

```
int x = 120;
int y = 60;
int radius = 12;

void setup() {
  size(240, 120);
  ellipseMode(RADIUS);
}

void draw() {
  background(204);
  float d = dist(mouseX, mouseY, x, y);
  if (d < radius) {
    radius++;
    fill(0);
  } else {
    fill(255);
  }
  ellipse(x, y, radius, radius);
}
```

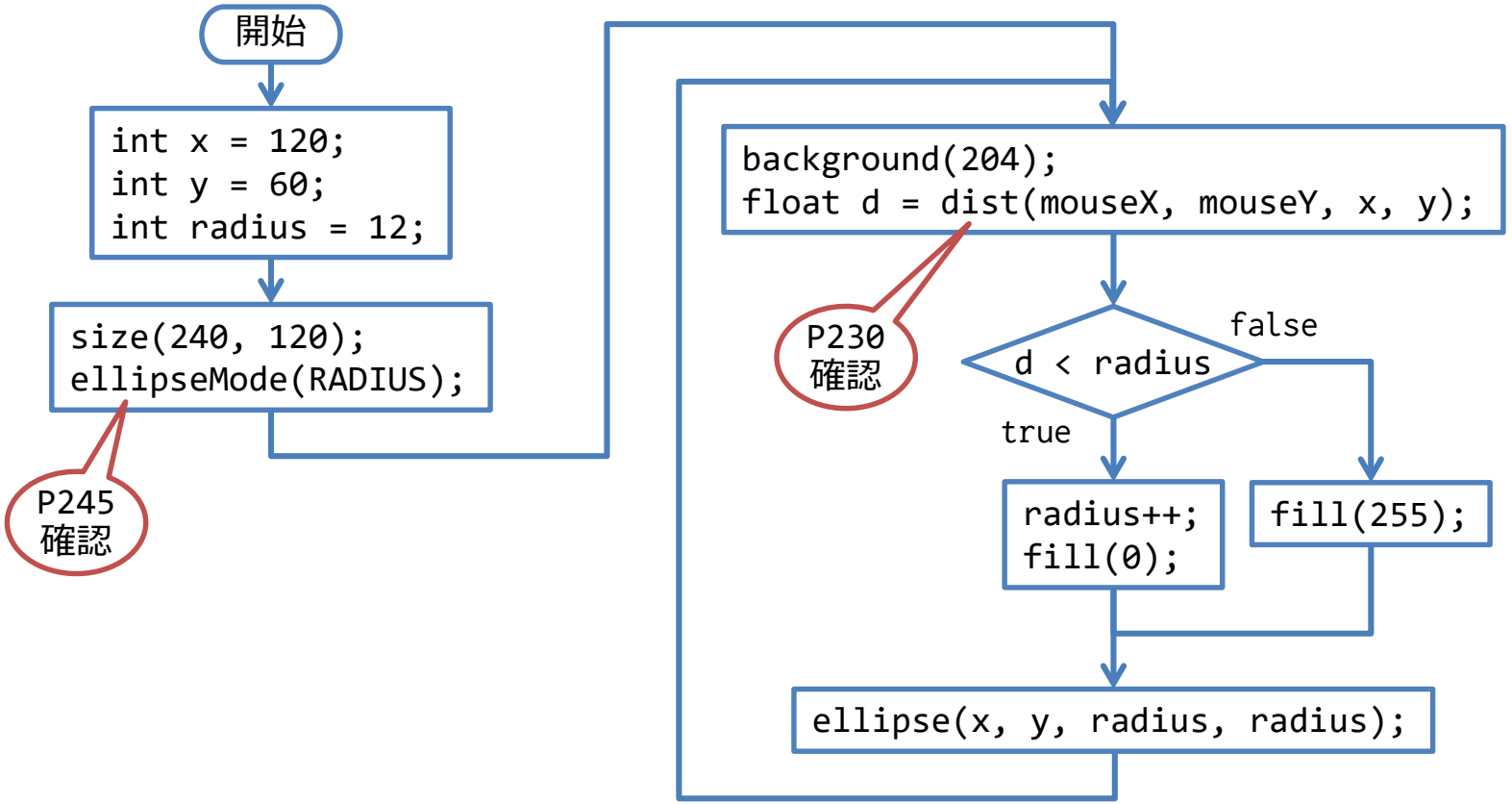




カーソルが円の
内部にある場合
黒い円が拡大

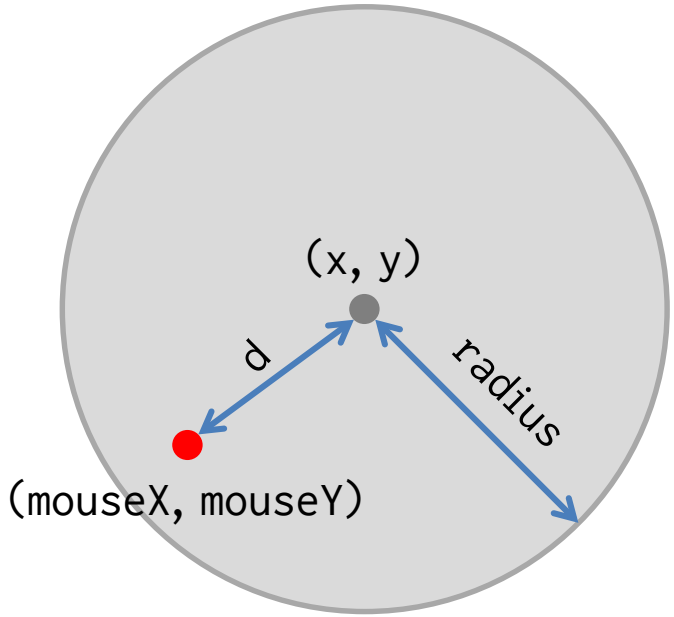
カーソル

- Example 5-14 円の境界



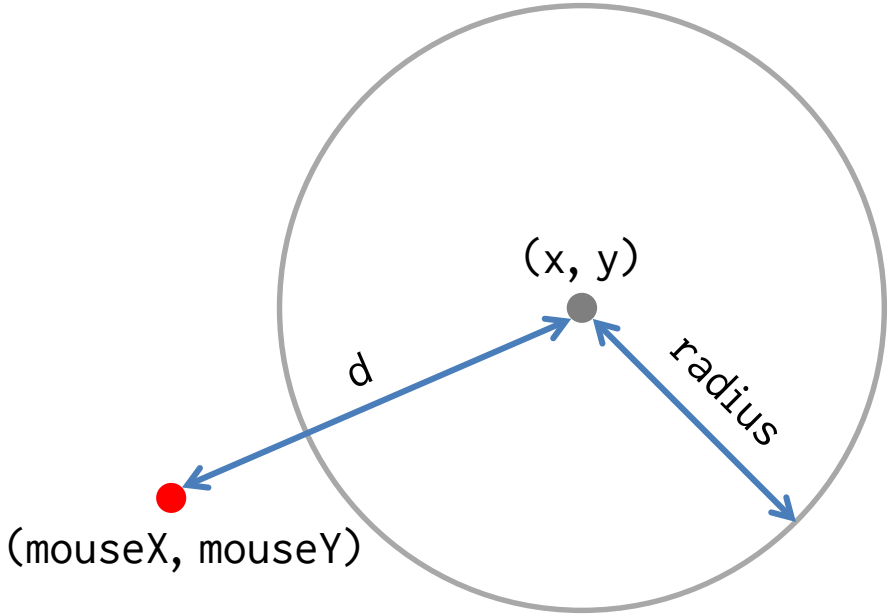
カーソルが円の内側にあるかどうか

カーソルが円の内側



距離 d が 半径 radius より小さい
条件 d < radius が真

カーソルが円の外側

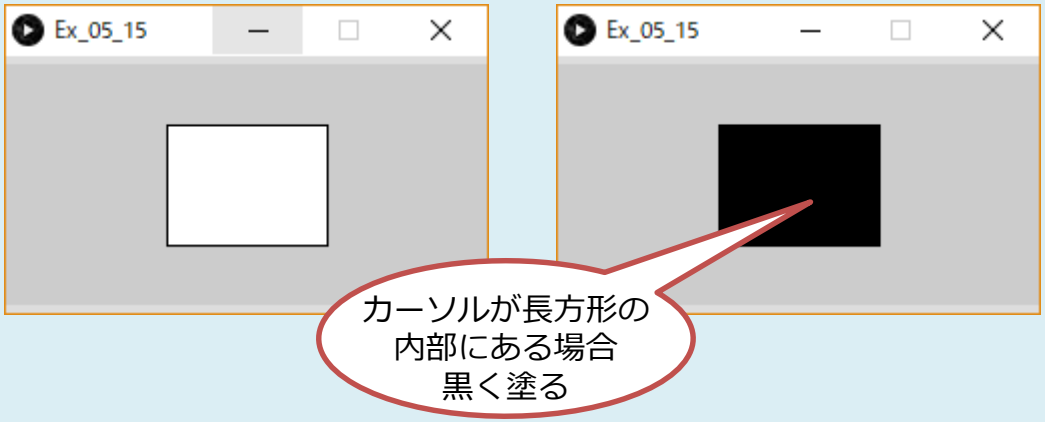


距離 d が 半径 radius より大きい
条件 d < radius が偽

カーソル

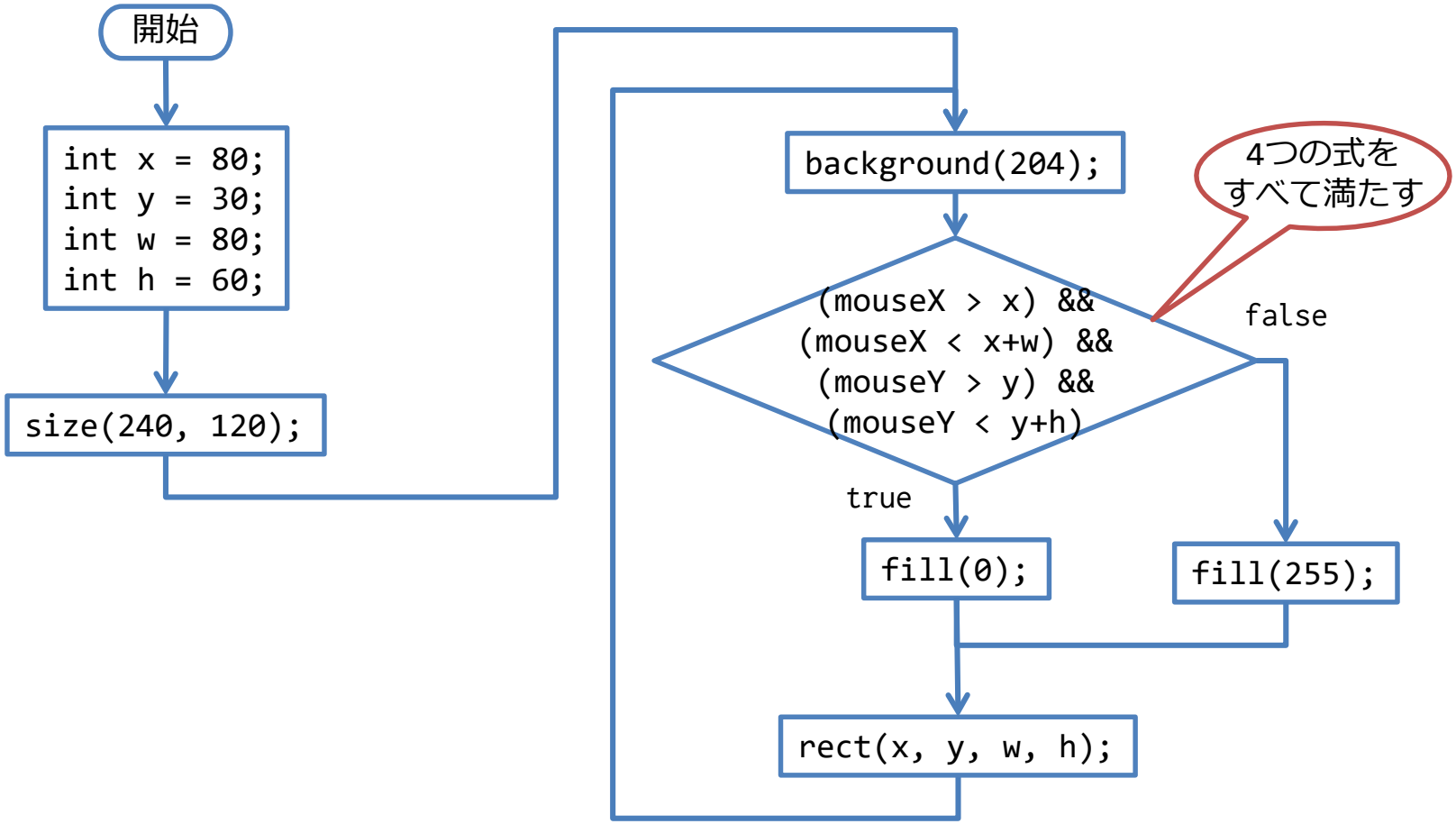
- Example 5-15 長方形の境界

```
int x = 80;
int y = 30;
int w = 80;
int h = 60;

void setup() {
  size(240, 120);
}

void draw() {
  background(204);
  if ((mouseX > x) && (mouseX < x+w) &&  (mouseY > y) && (mouseY < y+h)) {
    fill(0);
  } else {
    fill(255);
  }
  rect(x, y, w, h);
}
```



カーソル

- Example 5-14 円の境界

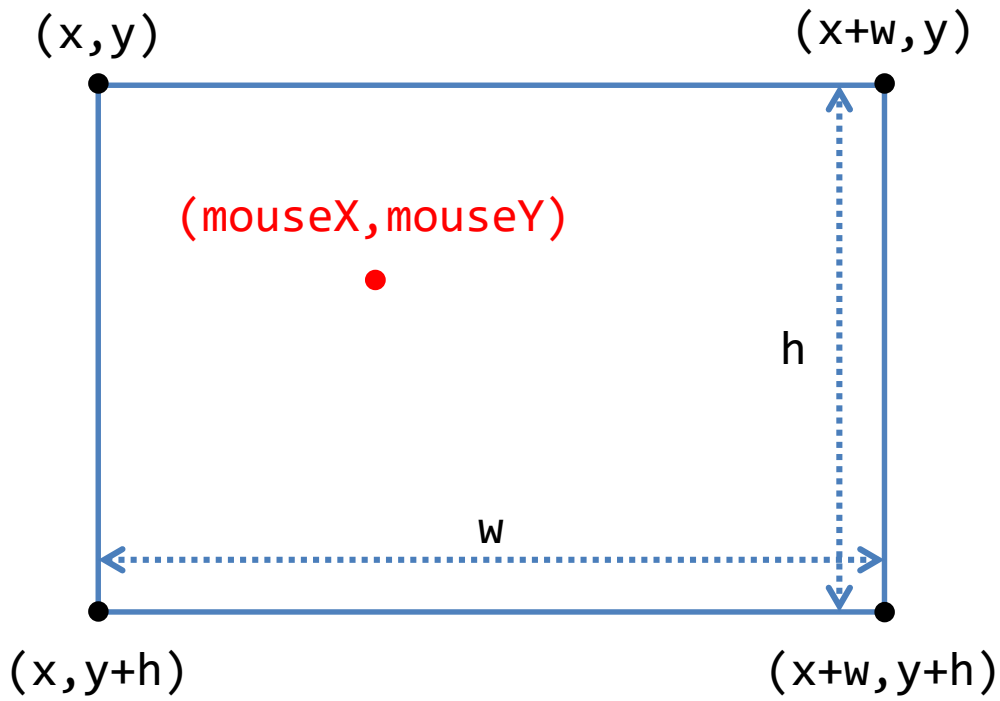


カーソルが長方形内部にあるかどうか

カーソル(mouseX, mouseY)が長方形内部にあるための条件

- mouseX が x より大きく、x + w より小さい
- mouseY が y より大きく、y + h より小さい

```
(mouseX > x) && (mouseX < x+w) && (mouseY > y) && (mouseY < y+h)
```



複数の式で条件をつなぐ場合、論理演算子を使う
P229参照

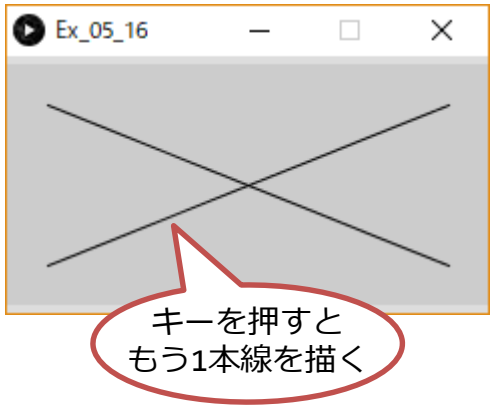
論理和 || または
論理積 && かつ

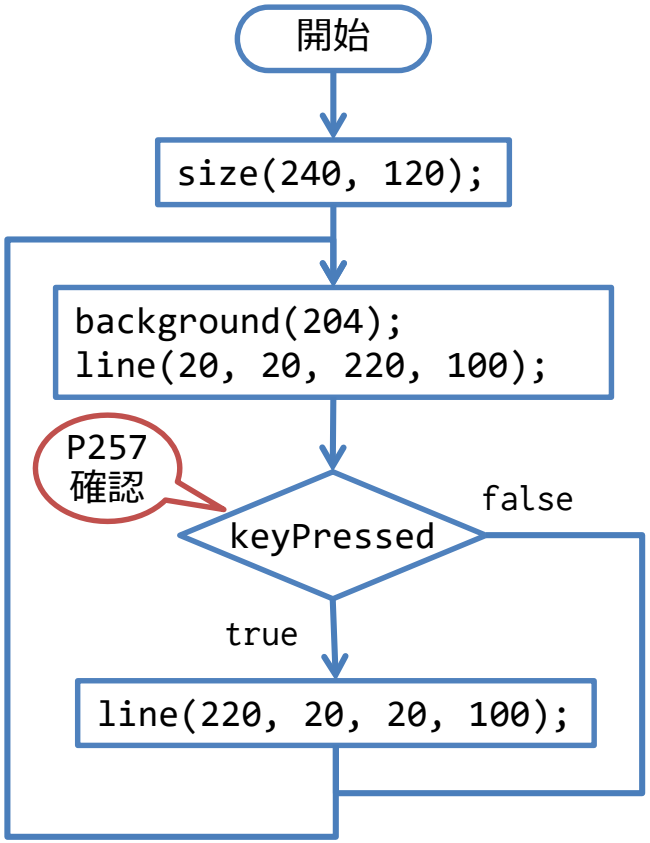
キーボードからの入力

- Example 5-16 キーを叩く

```
void setup() {
  size(240, 120);
}

void draw() {
  background(204);
  line(20, 20, 220, 100);
  if (keyPressed) {
    line(220, 20, 20, 100);
  }
}
```

keyPressed 変数 P257
キーを押した場合 true
押していない場合 false





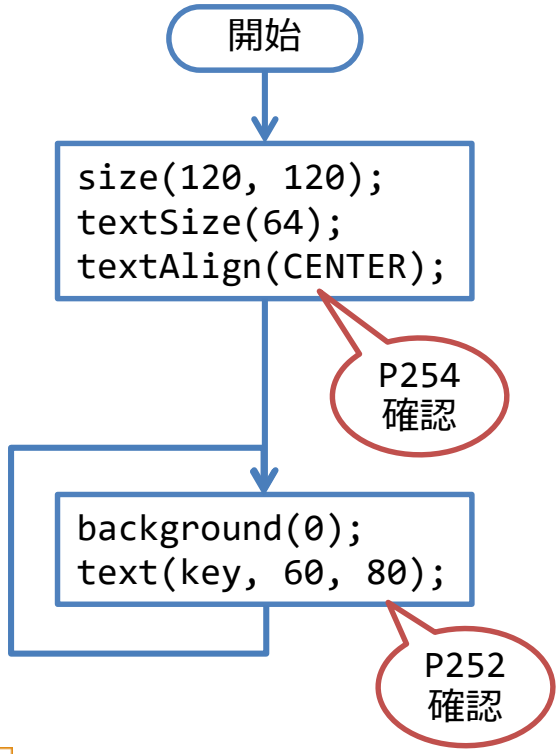
キーボードからの入力

- Example 5-17 文字を描く

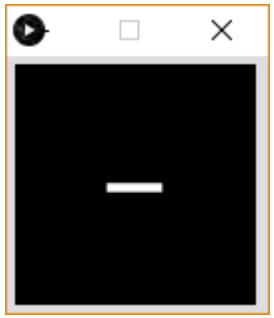
```
void setup() {
  size(120, 120);
  textSize(64);
  textAlign(CENTER);
}

void draw() {
  background(0);
  text(key, 60, 80);
}
```

key 変数 P257
最後に押したキーを保存
値は文字型 char

P251～254
textSize() 文字の大きさを設定
textAlign() 文字の揃え方を設定
text() 文字を表示する



半角英数
一部の記号を表示



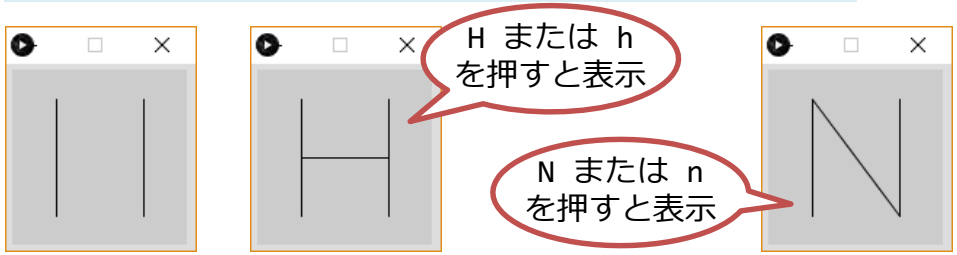
表示できない文字は
白い四角が表示

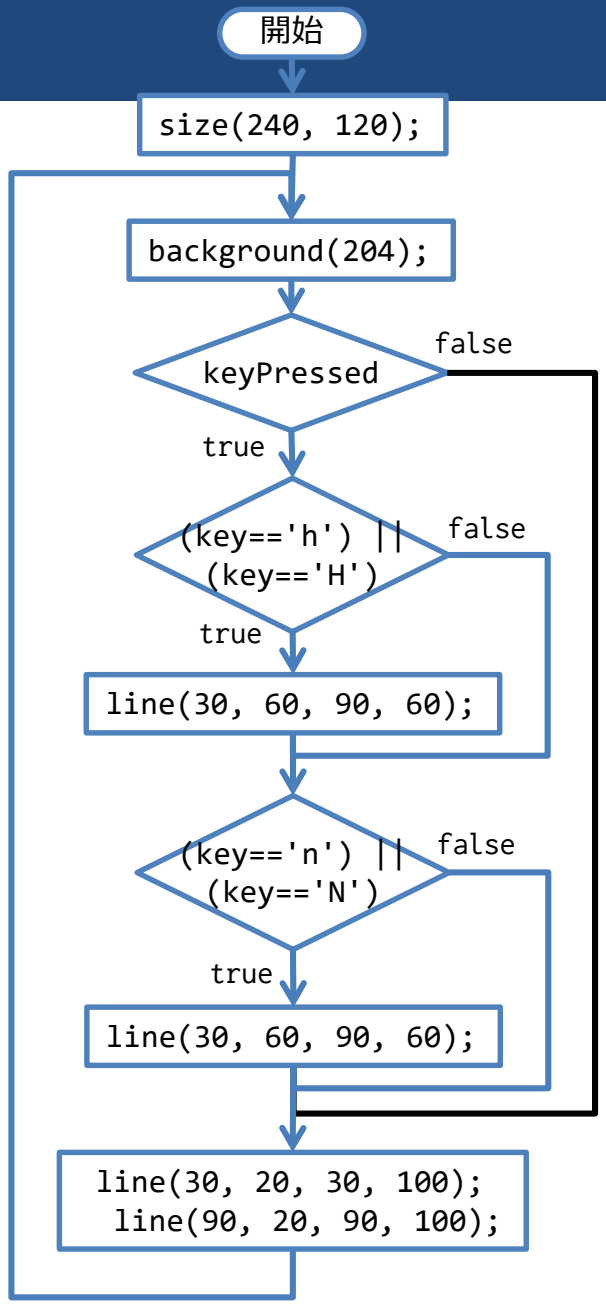
キーボードからの入力

- Example 5-18 特定のキーに反応する

```
void setup() {
  size(120, 120);
}

void draw() {
  background(204);
  if (keyPressed) {
    if ((key == 'h') || (key == 'H')) {
      line(30, 60, 90, 60);
    }
    if ((key == 'n') || (key == 'N')) {
      line(30, 20, 90, 100);
    }
  }
  line(30, 20, 30, 100);
  line(90, 20, 90, 100);
}
```

大文字/小文字
どちらも対応できる





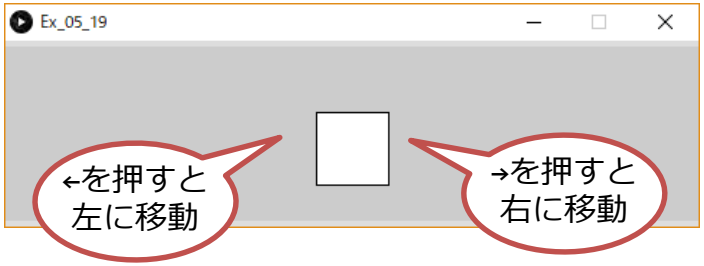
キーボードからの入力

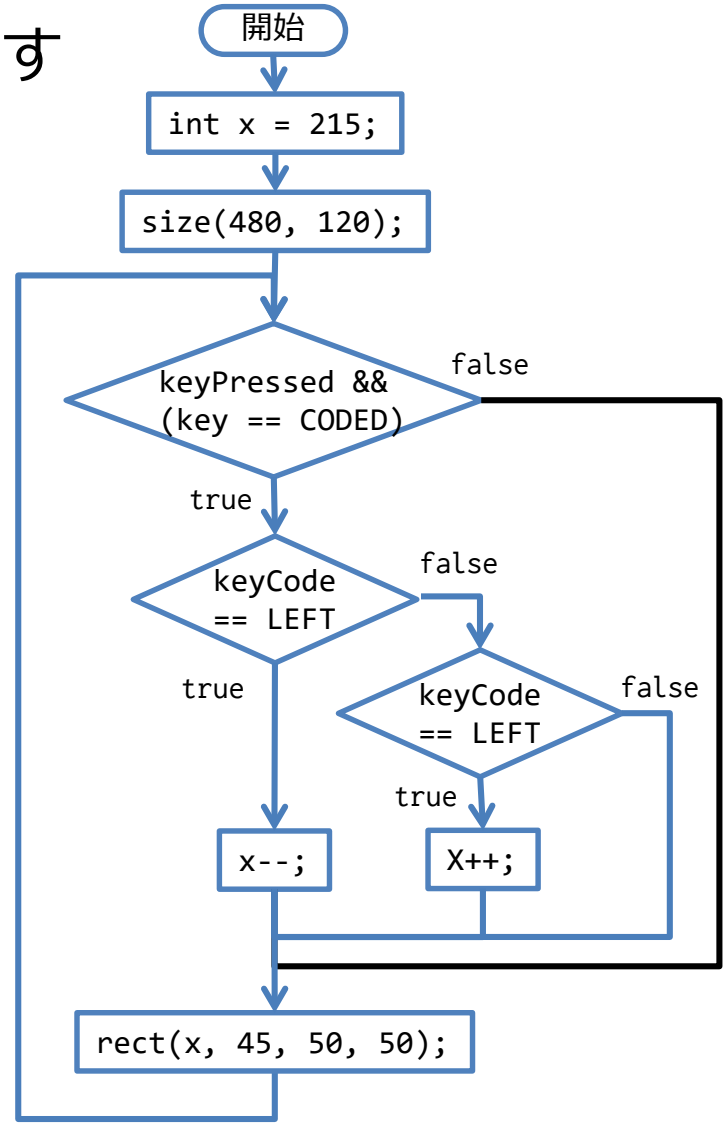
- Example 5-19 カーソルキーで動かす

```
int x = 215;

void setup() {
  size(480, 120);
}

void draw() {
  if (keyPressed && (key == CODED)) {
    if (keyCode == LEFT) { // 左矢印を押したら
      x--;
    } else if (keyCode == RIGHT) { // 右矢印
      x++;
    }
  }
  rect(x, 45, 50, 50);
}
```

特別なキーが押された場合

keycode
P257





CODED , keyCode変数 P257

- 最後に押したキーが特別なキーの場合、
  if (key == CODED) で確認、keyCode変数に保存
- keyCode変数　最後に押された特別なキーを保存
  - UP　↑（上矢印）キー
  - DOWN　↓（下矢印）キー
  - LEFT　←（左矢印）キー
  - RIGHT　→（右矢印）キー
  - ALT　Altキー
  - CONTROL　Ctrlキー
  - SHIFT　Shiftキー

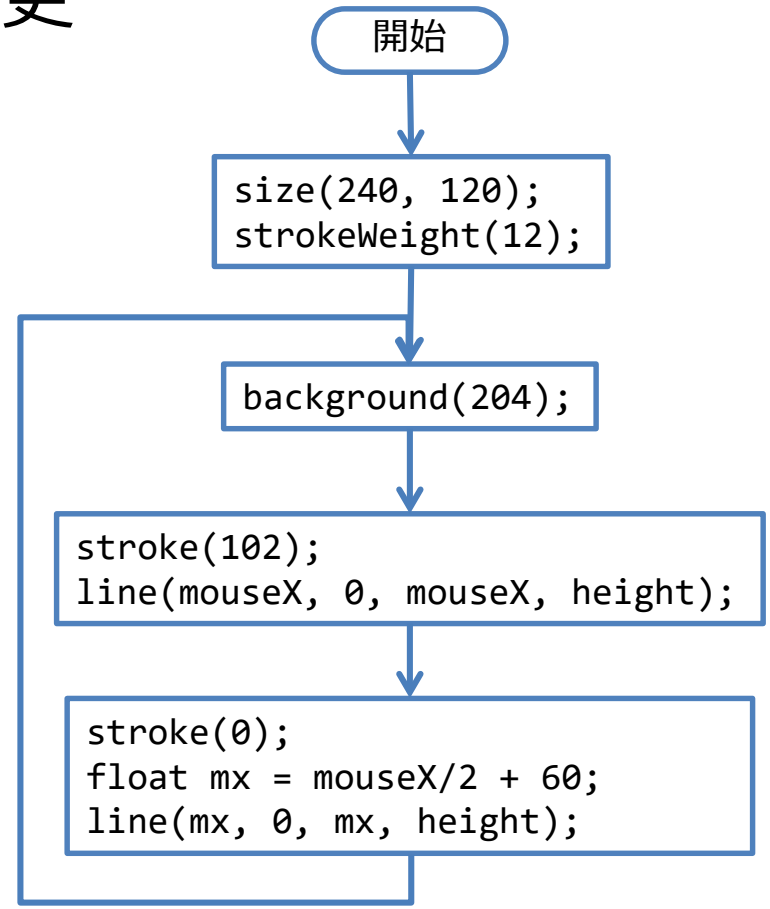
マッピング

- Example 5-20 値の範囲を変更

```
void setup() {
  size(240, 120);
  strokeWeight(12);
}

void draw() {
  background(204);
  stroke(102);
  line(mouseX, 0, mouseX, height);
  stroke(0);
  float mx = mouseX/2 + 60;
  line(mx, 0, mx, height);
}
```

計算して変換

灰色の線：変換なし
黒い線：値を変換



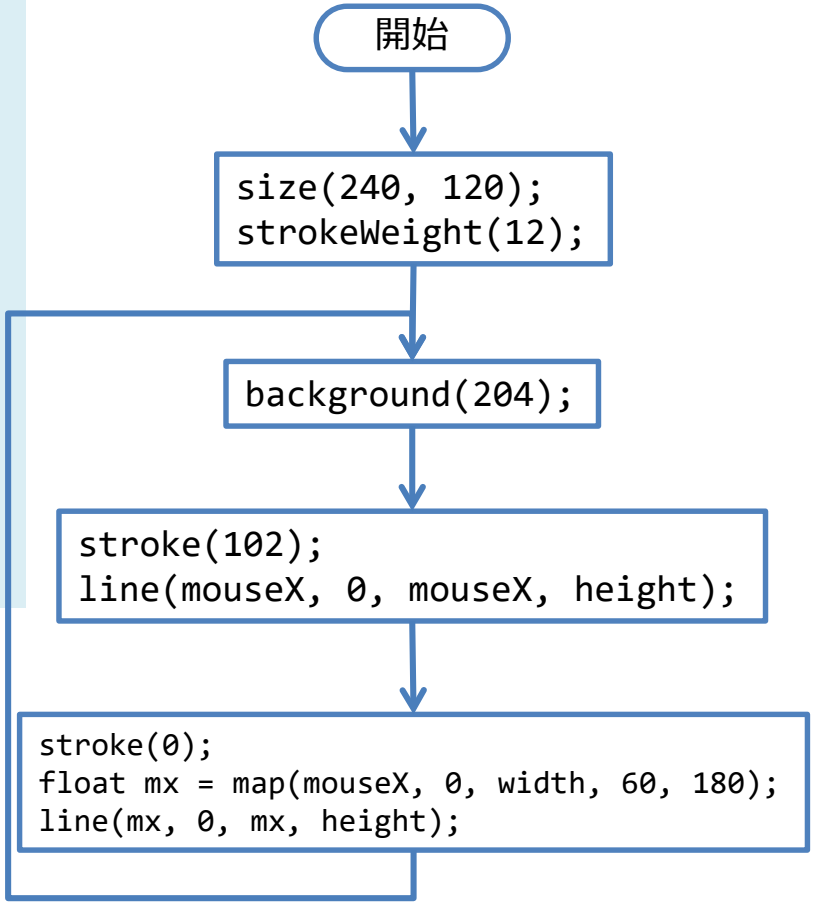
マッピング

- Example 5-21 map()関数でマッピング

```
void setup() {
  size(240, 120);
  strokeWeight(12);
}

void draw() {
  background(204);
  stroke(102);
  line(mouseX, 0, mouseX, height);
  stroke(0);
  float mx = map(mouseX, 0, width, 60, 180);
  line(mx, 0, mx, height);
}
```

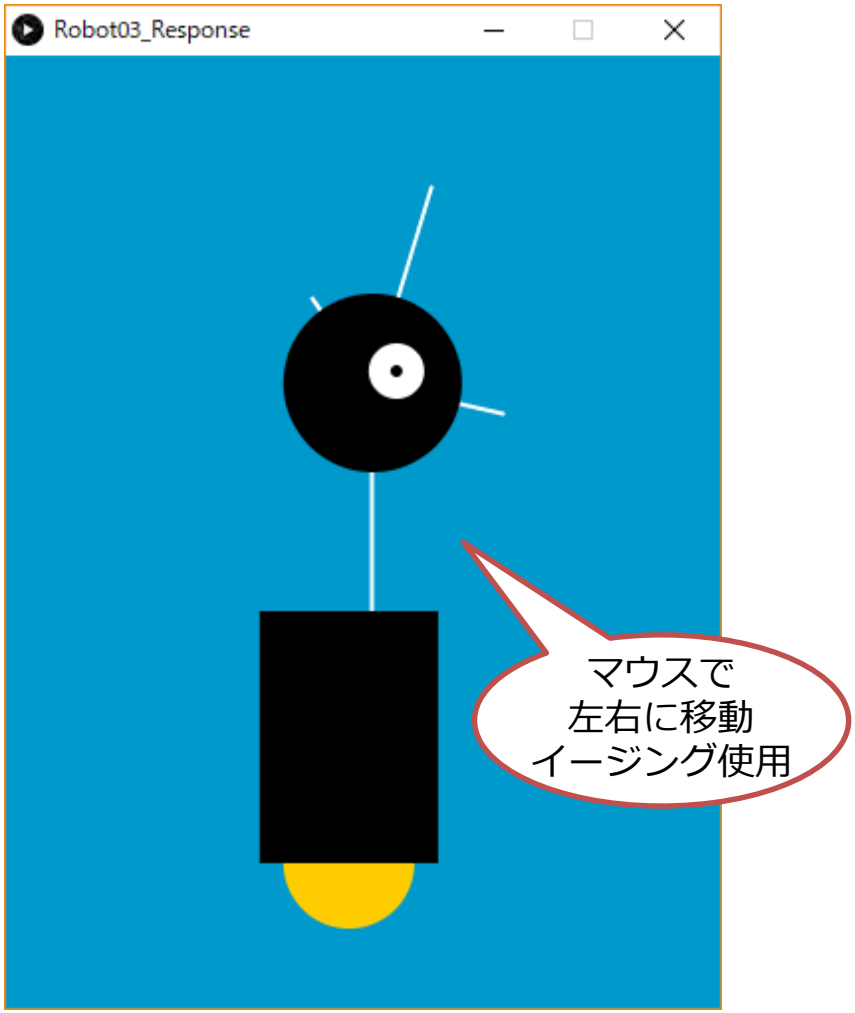
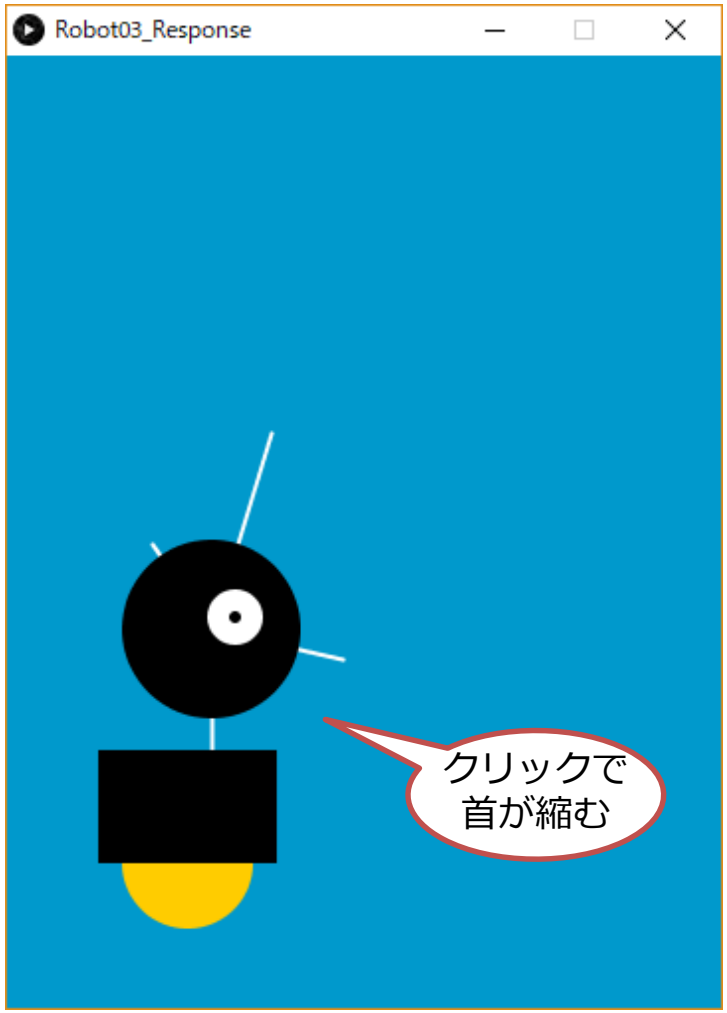
map()関数
P230

実行結果
Example 5-20 と同じ



Robot 3: Response

```
float x = 60;
float y = 440;
int radius = 45;
int bodyHeight = 160;
int neckHeight = 70;

float easing = 0.04;

void setup() {
  size(360, 480);
  ellipseMode(RADIUS);
}

void draw() {
  strokeWeight(2);
  int targetX = mouseX;
  x += (targetX - x) * easing;
  if (mousePressed) {
    neckHeight = 16;
    bodyHeight = 90;
  } else {
    neckHeight = 70;
    bodyHeight = 160;
  }
```

```
  float neckY = y - bodyHeight - neckHeight - radius;
  background(0, 153, 204);

  stroke(255);
  line(x+12, y-bodyHeight, x+12, neckY);

  line(x+12, neckY, x-18, neckY-43);
  line(x+12, neckY, x+42, neckY-99);
  line(x+12, neckY, x+78, neckY+15);

  noStroke();
  fill(255, 204, 0);
  ellipse(x, y-33, 33, 33);
  fill(0);
  rect(x-45, y-bodyHeight, 90, bodyHeight-33);

  fill(0);
  ellipse(x+12, neckY, radius, radius);
  fill(255);
  ellipse(x+24, neckY-6, 14, 14);
  fill(0);
  ellipse(x+24, neckY-6, 3, 3);
}
```

Robot 3: Response